

A detailed review of a radical-denesting program

DenestRadicals3, denest11, and their helpers

Algorithm, mathematical justification, defects, and repair strategy

September 5, 2026

Abstract

The supplied Wolfram Language program attempts to denest algebraic radical expressions by combining structural preprocessing with a search over multipliers. Its central experiment computes the minimal polynomial of a scaled root and intersects that polynomial with a binomial over an algebraic coefficient field. A lower-degree greatest common divisor may reveal a simpler representation. This is a mathematically meaningful strategy, and the program's denominator-rationalization identity is correct on its natural domain.

The implementation nevertheless has important correctness defects. It can lose the branch of the original expression, applies unguarded power identities in its custom factorizer, uses a heuristic zero test as a certificate, and contains a comparator that ignores its second argument. Its nominal multiplier cap is not a cap, invalid multipliers can lead to undefined arithmetic and unchecked indexing, and loading the file changes the protection of a system symbol. The article distinguishes these defects from heuristic incompleteness, representation choices, and merely unusual but valid language idioms. It supplies proofs, exact counterexamples, a source map, a defensive single-trial reference component, and a reproducibility package.

Evidence boundary. This is a complete source inspection supported by official language documentation and independent algebraic checks. The supplied program and the proposed Wolfram tests were *not* executed in a Wolfram kernel. Claims about mathematical rules are proved below; version-dependent evaluation traces are identified as regression tests rather than reported observations.

Contents

1	Scope, provenance, and overall assessment	3
1.1	What the program is trying to do	3
1.2	Three kinds of evidence	3
2	Prioritized findings	4
3	Architecture and public behavior	4
3.1	Function map	4
3.2	Calling conventions and their limitations	5
3.3	Preprocessing and task marking	5
3.4	Why tasks are combined	6
3.5	The two meanings of traversal	6
4	The mathematical engine	6
4.1	From an expression to a binomial equation	6
4.2	What a proper GCD proves	7
4.3	Why the roots of unity are appropriate	7
4.4	A direct successful case	8
4.5	A case where a multiplier helps	8
5	Multiplier search and its state machine	9
5.1	Initialization from visible terms	9
5.2	Degree history and outcome formats	9
5.3	Generating batches from discriminants	9
5.4	Why the discriminant is suggestive, not conclusive	10
5.5	Batch scheduling	10
6	The helpers: correct mathematics and unsafe reconstruction	10
6.1	Denominator rationalization is algebraically correct	10
6.2	What the nesting helpers measure	11
6.3	The custom factor representation	12
7	Correctness defects and counterexamples	12
7.1	F01: the original branch is discarded	12
7.2	F02: unguarded multiplication of nested exponents	14
7.3	F03: factoring a common base under a root	14
7.4	F04: a heuristic zero test is not a certificate	15

7.5	F05: zero multipliers and unchecked indexing	15
7.6	F09: the word “success” overstates the guarantee	16
8	Defects in search control, interfaces, and software engineering	16
8.1	F06: the prime comparator uses the wrong slot	16
8.2	F08: global protection and context pollution	17
8.3	F10: the input domain is not enforced	17
8.4	F11–F13: incompleteness, repeated work, and fragile state	17
8.5	F14: dead settings and ambiguous maintenance cues	18
9	Cost model and the false multiplier cap	18
9.1	F07: exact candidate counts	18
9.2	Other sources of growth	19
9.3	F12: absence of an application-level budget	19
10	Repair strategy and a conservative reference component	20
10.1	The order of repairs matters	20
10.2	Separate value certification from representation quality	20
10.3	A verified single-multiplier trial	20
10.4	Queue design and caching	21
11	Reproducibility and regression testing	21
11.1	Checks actually performed	21
11.2	Tests prepared for a Wolfram kernel	21
11.3	Further tests needed for a release	22
12	What should not be misdiagnosed	22
13	Conclusion	22
A	Complete supplied source	23
B	Defensive single-trial reference code	32
C	Unexecuted Wolfram regression contracts	34

1 Scope, provenance, and overall assessment

The object reviewed is the attachment `Pasted text.txt`, reproduced without editorial changes as `original.wl` in the accompanying archive. It contains 585 physical lines and 21,199 bytes. Its SHA-256 digest is

693a2825f1e2b8912e55371b90dc8ed115bc33ee707dd3a4521eb22bcd2ccc93.

All source-line references in this article refer to that file, not to the line numbers of the LaTeX source. Appendix A gives a numbered listing. The attachment supplies no test suite, package declaration, supported kernel version, formal input contract, or proof of completeness. No such information is inferred from the names of the functions.

1.1 What the program is trying to do

A representative objective is to replace

$$\sqrt{3 + 2\sqrt{2}} \quad \text{by} \quad 1 + \sqrt{2}, \quad \sqrt{5 + 2\sqrt{6}} \quad \text{by} \quad \sqrt{2} + \sqrt{3}.$$

Both changes preserve the number while reducing the nesting of explicit radicals. The implementation is more ambitious than a formula for nested square roots: it accepts rational powers, constructs algebraic multipliers, uses arbitrary-degree minimal polynomials, and explicitly enumerates roots of unity. Its architecture is best described as an *experimental, multiplier-guided algebraic simplifier*, not as a demonstrated complete denesting algorithm.

The main verdict is not that the algebraic idea is unsound. The main verdict is that **a promising search engine is wrapped in transformations and acceptance criteria that do not reliably preserve the original expression**. A search may be incomplete and still be useful; an exact simplifier must not silently return the wrong algebraic number.

1.2 Three kinds of evidence

Throughout the article, the following distinctions are maintained.

Source-established finding. A property follows directly from the supplied statements: for example, `Unprotect[Print]` is executed at load time and there is no matching restoration.

Mathematical counterexample or independent check. An identity, branch calculation, polynomial, or counting formula can be proved independently of the original evaluator. These checks were also exercised with Python, SymPy 1.14.0, and mpmath where appropriate.

Wolfram regression probe. A call intended for a real Wolfram kernel tests how automatic evaluation, factoring, pattern matching, and solver choices interact end to end. Such probes are included but are not presented as executed tests.

Official Wolfram documentation is used only to establish language behavior, not to replace the attached algorithm with a different one. In particular, principal powers, exact algebraic comparison, scoping, factoring options, and sorting semantics are checked against the documentation [2, 5, 6, 10, 8, 7].

2 Prioritized findings

The table is a navigation aid. “Critical” means an incorrect value can be accepted on a relevant branch or through an invalid transformation. “High” includes uncontrolled failure and potentially severe resource consumption. The severity of heuristic incompleteness depends on the promised interface; no completeness promise is present in the attachment.

ID	Priority	Finding	Source lines
F01	Critical	The candidate is checked against the principal root of <code>problem^r</code> , not against <code>problem</code> .	90–94, 121–134
F02	Critical	The custom factorizer merges nested exponents without a branch condition.	527–531
F03	Critical	Common-factor extraction under a noninteger outer power assumes an invalid general product-power identity.	538–543, 571–583
F04	High	Default <code>PossibleZeroQ</code> is used as an exact-equality certificate.	125–134
F05	High	Zero/invalid multipliers and an empty candidate set are not safely rejected.	107–134, 213–215
F06	High	The prime comparator compares the prime and exponent of its <i>first</i> argument.	177–194
F07	High	<code>multipliercap = 1000</code> does not bound candidate count or materialization cost.	162–199
F08	Medium	Loading the file leaves <code>Print</code> unprotected; helper symbols occupy the caller’s context.	1–4 and global definitions
F09	High	A reported success does not establish reduced radical depth, radical-only output, or final equality with the original input.	120–140, 309–323, 434–440
F10	Medium	Input-domain checks are incomplete and occur after global preprocessing in the wrapper.	5–12, 79–108, 443–454
F11	Medium	Search heuristics have no completeness certificate; partial factorization and seed normalization discard information.	159–247, 396–420
F12	High	There is no global visited-multiplier set or overall work budget; stack growth and expensive trials can dominate.	251–430
F13	Medium	Positional, mutable state and implicit outcome tags make invariants and error handling fragile.	105–106, 253–266, 292–309
F14	Low/medium	Dead settings, a five-pass factorization bound, and the meaning of <code>alllevels</code> can mislead maintenance and users.	95–102, 162–165, 398–420, 520–549

3 Architecture and public behavior

3.1 Function map

The twelve top-level definitions divide naturally into an orchestration layer, a search layer, and structural/algebraic helpers.

Function	Lines	Role
Strad	1–3	Calls the wrapper while dynamically suppressing Print .
DenestRadicals3	4–76	Finds and groups radical tasks, then invokes a chosen denester.
denest11	78–441	Builds multipliers, tests polynomial degrees and GCDs, and manages a search stack.
RationalizeDenominator	443–451	Computes an algebraic denominator’s inverse from its minimal polynomial.
AlgebraicQ	453–454	Tests membership in Algebraics .
nestdepths	456–465	Annotates an expression tree with a selected nesting count.
nestdepthsort	468–479	Groups those annotations by their nesting count.
maxnestdepth	482–497	Extracts nodes with maximal selected nesting.
ComplexitySort	500–504	Orders candidates by leaf count, then absolute value.
Factorc	506–516	Decodes the custom factor representation.
fullfactor	518–550	Produces that representation, recursively processing sums and powers.
combine	553–583	Extracts bases common to the terms of a sum.

Two important local functions, **trymultiplier** and **newmultipliers**, are defined inside **denest11**. The local **SumQ** belongs to the wrapper’s preprocessing rule.

3.2 Calling conventions and their limitations

The apparent interfaces are

```
Strad[arguments...]
DenestRadicals3[expr, alllevels : False, func : denest11]
denest11[problem, forcedmultipliers : 0]
```

The displayed colons summarize defaults; they are not replacement definitions. Only a literal **True** enables the **alllevels** branch. A value such as **Automatic** or a misspelling is treated like **False**, not rejected. The third argument is a callable denesting engine. A list supplied as the second argument to **denest11** supplies an initial multiplier batch; a non-list, including its default 0, selects heuristic initialization. Thus 0 and {0} have very different meanings.

The name **forcedmultipliers** should not be understood as a documented hard whitelist. Later improvements can enqueue generated batches even when the initial list was supplied by the caller. An empty supplied list yields no trials and leaves the problem unchanged. These are source-derived interface observations, not evidence that such behavior was intentional.

3.3 Preprocessing and task marking

The wrapper first evaluates **Factorc[expr]** *before* applying the algebraicity predicate. It then recognizes expressions of the form

```
radicand_?AlgebraicQ^pow_Rational
```

and temporarily replaces them by a local marker **f[radicand,pow]** (source lines 11–40). For each matched radicand it:

1. obtains a common coefficient using **FactorTermsList**;
2. factors the remaining part using the custom factorizer, and decomposes rational or Gaussian-rational coefficient factors;

3. combines factors that contain a sum or an explicit complex number with nonzero real part;
4. moves signs into a single sign accumulator, rationalizes denominators, and simplifies the reconstructed product.

The `SumQ` name is broader than “is a sum”: its test searches all levels for `Plus` or `Complex[Except[0],_]`. It is a heuristic classifier, not a mathematical characterization of additive structure.

The sign-flip loop deserves careful interpretation. Replacing a factor a_i by $-a_i$ and also multiplying an accumulated sign by -1 preserves the *product*. Consequently a numerical misclassification by itself does not prove that this loop changes the input value. The serious branch defects occur elsewhere, where fractional powers are reinterpreted or factored.

3.4 Why tasks are combined

Before calling the denester, the wrapper combines two marked factors when both radicands contain a `Power` node (source lines 43–47). It then changes `f[rad,pow]` to `f[rad^pow]`. A second rule combines one-argument markers when both enclosed numerical values have nonzero real and imaginary parts at a requested precision of ten digits (source lines 49–54).

These rules do not themselves assert $\sqrt{ab} = \sqrt{a}\sqrt{b}$. They form a marker around an already evaluated product. That grouping can be useful because a product may admit a joint simplification. It is nevertheless important: a product of principal roots need not lie in the sector of a principal root of the inferred degree. The grouped product can therefore violate an assumption that `denest11` silently makes; see Section 7.1.

3.5 The two meanings of traversal

With `alllevels != True`, the wrapper uses `ReplaceAll`, gathers distinct marked tasks, applies the chosen function once to each task, and simplifies the result. With `alllevels == True`, repeated marking exposes nested tasks, and a small dependency-resolution loop processes markers whose arguments contain no unresolved markers. Resolved tasks become replacement rules and are substituted into the remaining tasks (source lines 55–73).

This is approximately an innermost-first traversal of a dependency graph. It is *not* a proof that every radical newly introduced by the denester is reprocessed until no further improvement is possible. In particular, a fixed point of the marker-replacement rules is not the same thing as a fixed point of the complete denesting procedure. Wolfram’s `ReplaceRepeated` itself has a documented iteration limit by default; that built-in limit does not provide a useful whole-program search budget [13].

4 The mathematical engine

4.1 From an expression to a binomial equation

Let β denote the original problem presented to the core. The program chooses a positive integer r , called `reductionpower`, as the least common multiple of denominators attached to selected maximal-depth rational powers. It then sets

$$\text{outerpower} = \frac{1}{r}, \quad R = \text{radicand} = \beta^r.$$

A further product of denominator LCMs, called **extrapower**, is computed and printed, but is never used to change the actual search (source lines 90–102). Thus the “extra root” diagnostic does not mean that an extra-root algorithm is implemented.

For a proposed nonzero algebraic multiplier m , the trial routine forms

$$\alpha = (mR)^{1/r}, \quad p_m(X) = \text{MinPoly}_{\mathbb{Q}}(\alpha), \quad d_m = \deg p_m.$$

Here and throughout, a fractional power denotes the principal complex power. The one-argument form `MinimalPolynomial[alpha]` in the attachment is valid: it returns a pure function, so applying the returned value to `arg1` gives a polynomial [3].

When its degree-based screening condition is satisfied, the program computes

$$g_m(X) = \gcd(p_m(X), X^r - mR) \quad \text{over an algebraic coefficient field.} \quad (1)$$

The option `Extension -> Automatic` matters: it tells `PolynomialGCD` to include algebraic numbers appearing in the coefficients, rather than treating all displayed radicals as independent indeterminates [4]. The effective field can depend on the representations of those coefficients. It need not be described merely as $\mathbb{Q}(\sqrt{c})$ in more complicated examples.

4.2 What a proper GCD proves

Proposition 4.1. *Assume $r \geq 2$, $m \neq 0$, and R are exact algebraic numbers. Let K be a characteristic-zero field containing the coefficients used in (1). Then g_m has positive degree. Every root of g_m is a root of $X^r - mR$. If $\deg g_m < r$, the GCD gives an equation of lower degree for at least one admissible scaled root.*

Proof. The number $\alpha = (mR)^{1/r}$ is a common root of p_m and $X^r - mR$ in an algebraic closure of K . If the polynomials were relatively prime over K , Bezout’s identity would give $Ap_m + B(X^r - mR) = 1$ for suitable polynomials $A, B \in K[X]$. Evaluation at α would give $0 = 1$, a contradiction. Thus the GCD is nonconstant. Because it divides the binomial, each of its roots satisfies the binomial. The final assertion is precisely the inequality $\deg g_m < r$. $\square$

The proposition justifies the central computation. It does not say that solving the smaller equation produces a simpler *radical expression*. Its coefficients may already be complicated, and the solver may return `Root` objects or introduce additional radicals. The program conflates these different notions of progress.

4.3 Why the roots of unity are appropriate

For each root a returned by `Solve[g_m == 0, X]`, the program generates

$$\frac{a}{m^{1/r}} \zeta_r^k, \quad \zeta_r = e^{2\pi i/r}, \quad 0 \leq k < r. \quad (2)$$

This is a sound way to handle the phase introduced by scaling, provided the right original target is retained.

Theorem 4.2. *Let $m \neq 0$ and $\beta \neq 0$ be complex numbers, let $r \geq 2$ be an integer, and put $R = \beta^r$. For any a with $a^r = mR$, the set in (2) is exactly the set of all r roots of $X^r = R$. In particular, it contains β .*

Proof. Set $\mu = m^{1/r}$. Whatever principal phase is selected, $\mu^r = m$ and $\mu \neq 0$. Hence $(a/\mu)^r = R$. Multiplication by ζ_r^k preserves this equation. These r numbers are distinct because $a/\mu \neq 0$ and the roots of unity are distinct. A degree- r polynomial has at most r roots, so they are all the roots. Since $\beta^r = R$, β occurs among them. $\square$

No identity $(mR)^{1/r} = m^{1/r}R^{1/r}$ is needed in this proof. That identity is false in general. The phase enumeration is therefore a strength of the program, not a defect. Its defect is that it selects the branch using $R^{1/r}$ instead of β .

If the GCD has d roots, using every root and every phase produces dr formal candidates. For branch coverage, one GCD root already suffices by the theorem. Additional roots can still be useful as alternative symbolic representations, but their role should be distinguished from obtaining new algebraic values.

4.4 A direct successful case

Take $R = 3 + 2\sqrt{2}$ and $r = 2$. Since $1 + \sqrt{2} > 0$ and $(1 + \sqrt{2})^2 = R$, the desired principal root is $1 + \sqrt{2}$. With $m = 1$,

$$p_1(X) = X^2 - 2X - 1, \quad \gcd(p_1(X), X^2 - 3 - 2\sqrt{2}) = X - 1 - \sqrt{2}$$

over $\mathbb{Q}(\sqrt{2})$. The GCD has degree one, smaller than two, and the candidate selection can recover the answer immediately. These polynomial identities were independently checked exactly.

4.5 A case where a multiplier helps

Let

$$R = 5 + 2\sqrt{6} = (\sqrt{2} + \sqrt{3})^2.$$

The unscaled principal root has minimal polynomial

$$p_1(X) = X^4 - 10X^2 + 1,$$

and over $\mathbb{Q}(\sqrt{6})$ the GCD with $X^2 - R$ is the entire binomial. Thus this trial does not give a lower-degree equation. With $m = 2$, however,

$$(2 + \sqrt{6})^2 = 2R, \quad p_2(X) = X^2 - 4X - 2,$$

and the GCD becomes $X - 2 - \sqrt{6}$. Dividing the resulting root by $\sqrt{2}$ gives $\sqrt{2} + \sqrt{3}$.

The discriminant of p_1 is

$$\text{Disc}(p_1) = 147456 = 2^{14}3^2.$$

Consequently the discriminant-based generator has access to the primes 2 and 3 and can propose $m = 2$. This explains the heuristic's motivation without claiming a particular actual trial order: the comparator defect can affect that order.

5 Multiplier search and its state machine

5.1 Initialization from visible terms

Unless a list of multipliers is supplied, the core expands the rationalized radicand, strips visible rational coefficients from its terms, discards pure rational terms, and replaces certain complex terms by a signed imaginary unit (source lines 224–247). It then constructs complementary powers of visible radicals. For a factor displayed as $b^{a/q}$, a typical calculation is

$$\left(b^{a/q}\right)^{(q-a)/a}.$$

In a positive-real setting this is a way of suggesting a complementary factor $b^{(q-a)/q}$. In the complex setting it is only a candidate-generation heuristic, not a universally valid exponent simplification. Because these expressions are *multipliers to be tried*, changing their value is not by itself an incorrect rewrite of the original problem. It may instead make the search less effective.

The initialization also assumes that `List @@ Expand[...]` is a list of summands. That is valid when the head is `Plus`. If the expanded expression has head `Times` or `Power`, the same operation exposes factors or arguments instead. For an atom it does not magically produce a singleton list. A reliable implementation should explicitly use

```
terms = If[Head[expanded] === Plus, List @@ expanded, {expanded}];
```

The original nested-power precheck does not prove this missing summand invariant.

5.2 Degree history and outcome formats

A trial starts with a record of the form

$$\{\text{False}, \{m, p_m, d_m\}\}.$$

It may instead return

$$\{\text{True}, \{m, p_m, d_m\}\} \quad \text{or} \quad \{\text{True}, \{m, p_m, d_m\}, \text{answer}\}.$$

The first Boolean means “search progress,” not “a correct denesting was found.” In particular, a larger degree may be recorded as progress when it reveals that a previously observed smaller degree was significant (source lines 147–151).

The history starts with the sentinel $\{0, \infty, \infty\}$ and is maintained in a degree-based order. References such as `history[[-1]]` therefore mean a currently best degree record, not necessarily the most recent chronological attempt. The code’s use of GCDs of degrees can guide exploration, but degree arithmetic alone is not a certificate of field membership or denestability.

5.3 Generating batches from discriminants

For comparable observed degrees, `newmultipliers` partially factors the discriminant of a minimal polynomial, removes the sign factor, discards factors that fail `PrimeQ`, and tries products of selected primes with exponents between zero and $r - 1$. The command `FactorInteger[n, Automatic]` deliberately extracts only factors that are easy to find; an unresolved composite cofactor can remain [7]. Discarding that cofactor loses access to its unknown prime divisors.

There is an important qualification. Primes removed from the *combination prefix* by the ratio or length heuristic are added back as *individual* candidates through `Union[Divisors[...], primes]`. They are not all discarded outright. What is lost is their participation in most products, together with every prime hidden inside a discarded unresolved composite.

When the GCD of two observed degrees is smaller than either degree, the other branch simply proposes the product of the two recorded multipliers (source lines 201–207). The attachment supplies no theorem making this product sufficient. As a general warning against reasoning from degree GCDs alone, $\sqrt{2}$ has degree 2, $\sqrt[3]{3}$ has degree 3, and their product has degree 6, not degree $\gcd(2, 3) = 1$. This example is not asserted to be an execution trace of that particular branch; it illustrates why the heuristic needs a separate justification.

5.4 Why the discriminant is suggestive, not conclusive

For an algebraic integer α generating a number field K , the familiar relation

$$\text{Disc}(p_\alpha) = \text{Disc}(K)[\mathcal{O}_K : \mathbb{Z}[\alpha]]^2 \quad (3)$$

shows that a polynomial discriminant includes both field information and information about the chosen generator. To see the square, compare a power basis with an integral basis in the trace-pairing matrix: a basis change by a matrix A transforms the matrix to $A^T T A$, whose determinant is $(\det A)^2 \det T$. The absolute determinant of this lattice change is the index in (3).

For example, $\alpha = 3\sqrt{2}$ has minimal polynomial $X^2 - 18$ and discriminant $72 = 8 \cdot 3^2$, whereas the usual integral basis of $\mathbb{Q}(\sqrt{2})$ has discriminant 8. Thus the generator introduces the additional prime 3. For nonintegral generators one cannot apply (3) to a primitive, nonmonic polynomial without further qualification. In either setting, a list of discriminant primes is not automatically a complete or minimal list of useful denesting multipliers.

5.5 Batch scheduling

The stack stores either a concrete candidate list or a delayed call to `newmultipliers`. Some batches are marked “do full set.” Other batches may be interrupted when progress creates a more promising batch, with the untried remainder inserted separately (source lines 251–430). At the end of the initial heuristic pass, additional products of promising seeds are formed by `Distribute` and further exponent normalization.

This is a reasonable exploratory scheduling idea, but the representation is fragile. A single position may hold an unevaluated multiplier, an evaluated pair $\{m, d\}$, or a delayed generator specification, depending on the phase of the search. An answer is recognized using `ListQ[attempt[[-1]]] != True`, rather than an explicit outcome tag. That works only while the undocumented invariant “an answer is never a list” and the assumed attempt shape both hold.

6 The helpers: correct mathematics and unsafe reconstruction

6.1 Denominator rationalization is algebraically correct

Let $d \neq 0$ be algebraic and let

$$p(X) = a_0 + a_1 X + \cdots + a_s X^s$$

be its minimal polynomial over $\mathbb{Q}$, normalized in any nonzero rational multiple. Since $d \neq 0$, $a_0 \neq 0$. Dividing the identity $p(d) = 0$ by d gives

$$\frac{1}{d} = -\frac{a_1 + a_2d + \cdots + a_sd^{s-1}}{a_0}. \quad (4)$$

This is exactly the identity implemented in [source lines 443–450](#).

More explicitly, divide $p(X)$ by $X - d$ to obtain $p(X) = (X - d)q(X)$, because the remainder is $p(d) = 0$. Evaluating at zero gives

$$p(0) = -d q(0), \quad \frac{q(0)}{-p(0)} = \frac{1}{d}.$$

The source multiplies this last quotient by the original numerator. Its minus sign is correct, and a monic minimal polynomial is not required. The name `conjugate` is somewhat misleading: it stores a minimal polynomial, not one selected complex conjugate.

For $d = 1 + \sqrt{2}$, the polynomial is $X^2 - 2X - 1$, giving $d^{-1} = \sqrt{2} - 1$. For $d = 1 + i$, it is $X^2 - 2X + 2$, giving $d^{-1} = (1 - i)/2$. Both, along with three further examples, were checked by exact polynomial division in the accompanying Python script.

The limitations are contractual and computational. The helper does not guard against a zero or nonalgebraic denominator, does not constrain the cost of `MinimalPolynomial`, and does not first apply `Together`. A sum of several fractions can have top-level denominator 1 even though denominators remain in its terms. Some callers explicitly use `Together`, but the initialization at [source lines 224–229](#) does not. Expanding the inverse in (4) can also make an expression substantially larger.

6.2 What the nesting helpers measure

For a predicate P on expression nodes, the intended annotation follows the recurrence

$$h_P(E) = \mathbf{1}_{P(E)} + \max(\{h_P(c) : c \text{ is an argument of } E\} \cup \{0\}).$$

The source stores the original expression, its child annotations, and its height in nested lists. With $P(E)$ equal to “ E is a rational power,” this is a *syntactic radical height*. It is not an invariant of the algebraic number. The same number can have several heights, and a `Root` object may conceal substantial algebraic complexity while containing no displayed rational power at all.

`nestdepthsort` flattens this ad hoc annotation into height groups; `maxnestdepth` repeatedly prunes entries below the maximum height. The code is compact but depends on list shapes, `Sequence` splicing, and the distinction between expression data and structural lists. General list-valued input is not given a reliable independent contract. A cleaner implementation would return explicit records such as `{position, height}` and would handle the empty set of matching powers deliberately.

The early test in `denest11` is also broader than the later height test: it recognizes an outer `Power` whose base contains any `Power`, whereas the denominator computation selects only `_Rational` exponents. For example, a symbolic expression $(1+x^2)^3$ can satisfy the former test without containing a rational-exponent radical. The code does not establish that the resulting `reductionpower` is an integer at least two before using it. The trial helper is defined only for an outer exponent matching `Rational[1, _Integer]`, so an integer outer exponent 1 is not an interchangeable case.

6.3 The custom factor representation

`fullfactor` starts from `FactorList`, whose entries are factor–exponent pairs [14]. It recursively factors nonprime integer and rational bases, pulls exponents through displayed powers, and processes sums term by term using `combine`. The latter merges identical bases within each term, finds bases common to all terms, takes the minimum exponent for each common base, and subtracts it from every term.

`Factorrc` then reconstructs ordinary expressions through several shape-based rules, using the listability of `Times` and `Plus` and finally extracting the first item of a flattened result. This is not merely a cosmetic replacement for the built-in factorizer: it introduces new algebraic rewrite rules. Those rules require branch conditions that the implementation does not check.

7 Correctness defects and counterexamples

7.1 F01: the original branch is discarded

The central incorrect inference is

$$\beta \mapsto R = \beta^r \quad \text{followed by the assumption} \quad R^{1/r} = \beta.$$

For a principal root, this is false. Write $\beta = \rho e^{i\theta}$ with $\rho > 0$ and $-\pi < \theta \leq \pi$. There is an integer k such that

$$\text{Arg}(\beta^r) = r\theta - 2\pi k \in (-\pi, \pi].$$

Therefore

$$(\beta^r)^{1/r} = \beta e^{-2\pi i k/r}. \tag{5}$$

The reconstructed root equals β precisely when β lies in the principal root sector $-\pi/r < \text{Arg } \beta \leq \pi/r$. Neither the general core interface nor the wrapper’s product-grouping rules establish this condition. Principal-value semantics are the documented behavior of `Power` [2].

The source’s final filter is

```
Select[possibilities, PossibleZeroQ[# - radicand^outerpower] &]
```

The expression after the minus sign should be the *original problem*, not its reconstructed principal root. Merely replacing `PossibleZeroQ` with an exact method would not fix this wrong target.

7.1.1 An exact negative-real counterexample

Take the core problem

$$\beta = -\sqrt{3 + 2\sqrt{2}} = -(1 + \sqrt{2}), \quad r = 2.$$

The trial path, with multiplier $m = 1$, has

$$R = 3 + 2\sqrt{2}, \quad p(X) = X^2 - 2X - 1, \quad g(X) = X - 1 - \sqrt{2}.$$

Its phase candidates are $1 + \sqrt{2}$ and $-1 - \sqrt{2}$. The code compares them to $R^{1/2} = 1 + \sqrt{2}$ and selects the *positive* candidate. The original negative candidate is the one it should preserve. This is an exact sign error, not a numerical-precision issue.

The corresponding regression call is

```
denest11[-Sqrt[3 + 2 Sqrt[2]], {1}]
```

The algebraic trial path above is proved and independently checked. The call itself remains an unexecuted Wolfram regression probe; automatic evaluation can affect whether a written nested expression survives to enter that path. That qualification does not make the normalization identity valid.

7.1.2 A complex product relevant to the wrapper

The defect is not restricted to a caller deliberately putting a minus sign in front of a core problem. Define

$$u = 1 + i\sqrt{2}, \quad v = 1 + i\sqrt{3}.$$

Since both have positive real part,

$$\sqrt{-1 + 2i\sqrt{2}} = u, \quad \sqrt{-2 + 2i\sqrt{3}} = v.$$

Their product is

$$\beta = uv = 1 - \sqrt{6} + i(\sqrt{2} + \sqrt{3}), \tag{6}$$

which has negative real part and positive imaginary part. Hence $\sqrt{\beta^2} = -\beta$. Explicitly,

$$R = \beta^2 = 2 - 4\sqrt{6} - i(4\sqrt{2} + 2\sqrt{3}).$$

The reconstructed principal root $\gamma = -\beta$ has minimal polynomial

$$p_\gamma(X) = X^4 + 4X^3 + 4X^2 + 48X + 144,$$

and over a coefficient field containing the displayed radicals,

$$\gcd(p_\gamma(X), X^2 - R) = X - \sqrt{6} + 1 + i\sqrt{2} + i\sqrt{3} = X - \gamma.$$

The source again filters for γ , not for β .

This example lies in exactly the kind of product class that the wrapper's marker-combination rules create. The preceding calculation establishes the core branch defect for the grouped algebraic value. The public-wrapper probe in the archive tests the additional interaction with **Factoric** and automatic evaluation; no unobserved wrapper transcript is claimed.

7.1.3 Required repair

Pass the original target explicitly into every trial, and select by a certified equality to that target. The roots-of-unity enumeration already contains the correct phase under the hypotheses of Theorem 4.2. The repair does not require guessing a sign from approximate real parts. For an exact algebraic target, a minimal acceptance pattern is

```
Select[candidates, TrueQ[RootReduce[# - original] == 0] &]
```

This must be accompanied by input, failure, and empty-list handling.

7.2 F02: unguarded multiplication of nested exponents

At source lines 527–531, a factor record whose base is itself a power is changed from the mathematical shape

$$\{a^b, c\} \text{ to } \{a, bc\}.$$

That represents the rewrite $(a^b)^c \mapsto a^{bc}$ without a branch test. The rewrite is not valid for general principal powers. For example,

$$\sqrt{(-2)^2} = 2 \neq -2.$$

For a symbolic input, the normalization can move through the conceptual records

$$\{\sqrt{x^2}, 1\} \mapsto \{x^2, 1/2\} \mapsto \{x, 1\}.$$

The result x does not equal $\sqrt{x^2}$ on the real line: the correct real-valued expression is $|x|$. The regression suite therefore checks `Factorc[Sqrt[t^2]]` and only *afterward* substitutes $t = -2$. Substituting before the call could allow ordinary evaluation to hide the faulty symbolic rule.

The outer wrapper makes this especially important. Its first operation is `Factorc[expr]`, before the `AlgebraicQ` pattern restriction. Thus a user can pass symbolic input that is altered unsafely rather than rejected or left unchanged. Even if the intended public domain is only explicit algebraic constants, the implementation does not enforce that domain before applying the problematic rewrite.

A safe repair needs a theorem for the permitted case: for instance, a positive real base with suitable real exponents, or an integer outer exponent. Otherwise the nesting must be retained. A blanket call to `PowerExpand` would repeat the same fundamental mistake rather than repair it.

7.3 F03: factoring a common base under a root

The common-factor logic of `combine` is valid for a sum before an outer power is taken. It becomes unsafe when a common factor is assigned exponent `factor*pow` outside the residual sum. The resulting identity is

$$(cS)^q \stackrel{?}{=} c^q S^q,$$

which is not generally true for noninteger q .

For a concrete symbolic example, factoring the common x in $x + x^2$ suggests

$$\sqrt{x + x^2} \stackrel{?}{=} \sqrt{x}\sqrt{1 + x}.$$

At $x = -2$, the left side is $\sqrt{2}$, whereas the right side is $(i\sqrt{2})i = -\sqrt{2}$. This is the behavior targeted by the `Factorc[Sqrt[t + t^2]]` substitution probe. The formal common-base records for the two terms are $\{\{x, 1\}\}$ and $\{\{x, 2\}\}$, so the source extracts exponent 1 before multiplying it by the outer exponent $1/2$.

The phase issue also occurs for explicit algebraic complex numbers. Put

$$B = \sqrt{2} + (-1)^{1/3} = \sqrt{2} + \frac{1}{2} + \frac{\sqrt{3}}{2}i.$$

Then $0 < \text{Arg } B < \pi/2$, so

$$\sqrt{-B} = -i\sqrt{B}, \quad \sqrt{-1}\sqrt{B} = i\sqrt{B}.$$

Extracting a common -1 from the terms under this root reverses the sign. This proves that restricting coefficients to algebraic numbers, without a positivity or branch condition, does not justify the rule.

The repair is to keep factored products *inside* the noninteger power unless a valid branch condition is established. Positive real factors can be extracted safely for real rational exponents, subject to ordinary nonzero domain conditions for negative exponents. General complex factors require explicit phase accounting or exact comparison against the unsplit original.

7.4 F04: a heuristic zero test is not a certificate

Default `PossibleZeroQ` uses symbolic and numerical heuristics. The official documentation explicitly describes possible incorrect positive answers, including examples involving explicit algebraic quantities perturbed by a very small nonzero rational number. It documents `Method -> "ExactAlgebraics"` for guaranteed decisions on explicit algebraic numbers [5].

Therefore the current filter cannot be advertised as an exact correctness certificate. A false positive may admit a wrong candidate; failure to recognize a true zero can also leave no acceptable candidate. Increasing a numerical precision elsewhere in the code does not turn this test into a proof.

For the intended exact algebraic domain, a conservative certificate is `RootReduce[candidate - original] == 0`. The alternative exact mode of `PossibleZeroQ` is also relevant, but it must be applied to the *correct original target*. A timeout, unevaluated result, or unsupported input must mean “not certified,” not “approximately equal, therefore accepted.” This is a local algebraic equality decision, not a claim that arbitrary symbolic equality is decidable.

7.5 F05: zero multipliers and unchecked indexing

The multiplier interface accepts an arbitrary list without checking that its elements are nonzero exact algebraic numbers. Consider a quadratic trial with $m = 0$. Then

$$\alpha = 0, \quad p(X) = X, \quad \gcd(X, X^2) = X.$$

The GCD passes the source’s lower-degree test. Solving gives $a = 0$, after which (2) attempts to divide by $0^{1/2}$, producing undefined arithmetic. The following supplied-input probe therefore has a definite invalid path:

```
denest11[Sqrt[3 + 2 Sqrt[2]], {0}]
```

The precise subsequent messages and returned symbolic debris depend on kernel error propagation, and are not claimed here as observed output.

Independently of this trigger, the accepted candidates are stored in `result`, after which the source accesses `result[[1]]` without first checking whether the list is empty (source lines 125–134). An empty list may follow invalid arithmetic, an unexpected solver result, a failed equality test, or an interrupted algebraic computation. A proper guard is mandatory:

```
If[valid == {},  
  Return[Failure["NoCertifiedCandidate", <||>]]  
];
```


Every proposed multiplier must be rejected before division unless it is a supported, certified nonzero value. The problem $\beta = 0$ should be handled separately as a trivial case.

7.6 F09: the word “success” overstates the guarantee

The source declares a denesting when the GCD degree drops and an accepted candidate is returned. It does not enforce any of the following independent properties:

1. equality to the original expression before preprocessing;
2. a decrease in explicit radical nesting or another chosen complexity measure;
3. output in a specified radical-only expression language.

A lower-degree polynomial equation need not yield fewer nested radicals. Furthermore, `Solve` can return `Root` objects; for example, its documented default treatment of numerical cubic equations can use such objects [18]. Hiding an algebraic number inside `Root` is not the same task as denesting its radical representation.

The final comparison `solution != problem` is a structural comparison, not a semantic equality test or a simplicity ordering. Multiple retained candidates are likewise not necessarily different solutions: ordinary `DeleteDuplicates` can leave distinct expressions for the same algebraic number. If every candidate is exactly equal to the target, they are all the same *value*. Select the best representation by an explicit metric, then deduplicate with exact algebraic equality if needed.

8 Defects in search control, interfaces, and software engineering

8.1 F06: the prime comparator uses the wrong slot

The comparator in [source lines 181](#) is

```
#1[[1]] <= #[[2]] &
```

In a pure function, `##` means `##1`, not `##2` [9]. If the first factorization pair is $\{p, e\}$ and the second is $\{q, f\}$, the predicate evaluates $p \leq e$. It never examines q or f . For example,

$$\text{bad}(\{2, 1\}, \{3, 1\}) = \text{False}, \quad \text{bad}(\{3, 1\}, \{2, 1\}) = \text{False}.$$

It therefore does not order the primes. With a first pair $\{2, 18\}$, the same predicate is true no matter what the second pair is.

This affects more than presentation: subsequent prime-ratio checks and prefix selection rely on ascending prime order. The most direct repair is

```
primes = SortBy[Select[primes, PrimeQ[First[#]] &], First];
```

Alternatively, the intended binary comparator is `##1[[1]] <= ##2[[1]] &`. The other uses of `<=` in sorting should not be condemned merely because another programming language requires a strict weak comparator; Wolfram’s sorting interface explicitly supports Boolean ordering predicates [8]. The demonstrated defect is the wrong slot.

8.2 F08: global protection and context pollution

The second line of the attachment executes `Unprotect[Print]`. No later line restores that attribute. The dynamic binding in

```
Block[{Print = (Null &)}, DenestRadicals3[eaf]]
```

restores the temporarily localized *value* on leaving the block, but it does not undo the separate earlier removal of `Protected`. The `Unprotect` operation changes attributes; `Block` localizes values [12, 10]. Thus merely loading this file changes the global session's protection state.

A preferable design routes diagnostics through a private logger or a `Verbose` option and never alters `System'Print`. Suppressing `Print` is also not the same as suppressing kernel messages, and arguments to `Strad` ordinarily evaluate before its body is entered.

The absence of `BeginPackage` and a private context also matters. Loading the file clears and redefines generic names such as `combine`, `Factorrc`, and `AlgebraicQ` in the active context. This is normal for an isolated notebook experiment but risky for a reusable package. In addition, dynamic `Block` localization does not create fresh marker symbols. A caller's attributes on a symbol such as `f` can matter. `Module` creates fresh symbols and is a better default for private implementation state [11]. This is not a claim that every use of `Block` in the source is wrong.

8.3 F10: the input domain is not enforced

The mathematical core needs at least explicit exact algebraic inputs, nonzero multipliers, and an integer $r \geq 2$. The public definitions accept much more. They do not provide consistent behavior for approximate numbers, symbolic parameters, transcendental constants, list-valued inputs, invalid options, or malformed trial outcomes.

`AlgebraicQ[num_] := Element[num, Algebraics]` is not automatically a bug just because `Element` may remain symbolic. A `PatternTest` accepts only literal `True`; an undecided predicate fails the match [15, 16]. The genuine problems are that the predicate is not a full API contract and is applied after the global custom factorizer. The direct core has still fewer guards.

For a deliberately numerical algebraic interface, an initial predicate can combine `NumericQ`, infinite `Precision`, and a literal algebraic membership result. Using precision also catches inexact complex atoms; merely searching for visible `_Real` subexpressions is not a complete precision policy. A separate symbolic interface would need explicit assumptions and assumption-preserving transformations. `ComplexExpand` assumes unlisted variables are real [17]; that is another reason not to pass unrestricted symbolic expressions through this numeric preprocessing pipeline.

8.4 F11–F13: incompleteness, repeated work, and fragile state

The search is not proved exhaustive. It screens trials by minimal-polynomial degree, uses visible-term guesses, partially factors discriminants, limits prime combinations, and normalizes away some seed factors. A failure to find an improvement proves only that the selected search did not find one. It is not a certificate that no denesting exists.

The stack also has no global set of previously tried exact multipliers. Different generated batches can contain the same multiplier, and structurally different expressions can denote the same multiplier. A local `DeleteDuplicates` or `Union` does not provide global semantic memoization. Repeating

`MinimalPolynomial` and `PolynomialGCD` for these values can be very expensive.

The state representation compounds the problem. Positional indexing such as `multipliers[[1, 2, {1, -1}]]` assumes several nested layouts at once. The same stack field changes interpretation as candidates are evaluated. The Boolean named `stackchange` is set when progress is noticed, not solely at a physical stack insertion. The logic is difficult to audit and requires tests of interruption and resumption invariants. No particular infinite loop or universal stack-corruption trace is asserted here; the source simply lacks a clear, enforced state contract.

Use named records with distinct variants: an unevaluated candidate, an attempt result, a pending generator, and a queue batch. Suggested trial outcomes are `"NoProgress"`, `"VerifiedCandidate"`, `"InvalidInput"`, `"TimedOut"`, and `"InternalFailure"`. “Found an expression” and “made a search-scheduling improvement” should not share a Boolean flag.

8.5 F14: dead settings and ambiguous maintenance cues

Several details are demonstrable data-flow shortcomings rather than mathematical counterexamples. The computed `extrapower` is only printed. `maxprimes = 14` is never read. The local `best` assigned in the post-initial-pass block is unused, whereas `worst` is used. Some local names and comments describe abandoned variants, and substantial obsolete code remains commented out.

The `j < 5` bound in `fullfactor` is a real implementation limit. It prevents an unlimited local normalization loop, but there is no fixed-point check or status reporting when the fifth pass still changes the representation. It should not be described as a proof of complete factoring. Conversely, the existence of this bound means it would be inaccurate to claim that every loop in the program is unbounded.

9 Cost model and the false multiplier cap

9.1 F07: exact candidate counts

Let t distinct primes be selected for combination and suppose $r \geq 2$. The integer fed to `Divisors` is

$$N = \left(\prod_{j=1}^t p_j \right)^{r-1}.$$

Every divisor has the unique form

$$\prod_{j=1}^t p_j^{e_j}, \quad 0 \leq e_j \leq r-1,$$

so N has exactly r^t divisors. Dropping the unit leaves $r^t - 1$ candidates. If there are s detected primes in all, with the other $s - t$ reintroduced only as singletons, the actual union has $r^t - 1 + (s - t)$ candidates before scaling. For a nonzero fixed multiplier, scaling does not identify distinct values.

When enough primes survive the prefix heuristics, the source uses

$$t = \max \left(5, \left\lceil \frac{\log 1000}{\log r} \right\rceil \right).$$

The ceiling, the minimum of five selected primes, and the later singleton union are all incompatible with treating 1000 as a hard cap.

Root degree r	Selected primes t	$r^t - 1$
2	10	1,023
3	7	2,186
4	5	1,023
5	5	3,124
6	5	7,775
10	5	99,999
100	5	9,999,999,999

These are conditional counts assuming the indicated number of primes is available. They are not measured timings or assertions that every input produces that many candidates. The final row nevertheless shows the scale of the missing safeguard.

A cap must be enforced *before* candidate materialization. Computing all `Divisors` and then taking the first 1000 does not prevent the allocation. A bounded iterator over exponent vectors, preferably ordered by a useful cost, is a more appropriate generator.

9.2 Other sources of growth

The post-initial-pass `Distribute` operation forms products by taking or not taking selected seed multipliers. With k such choices, the unpruned product family can have 2^k members. Subsequent normalization may collapse many of them, but it does not remove the cost of forming the intermediate family.

For an individual trial, the important parameters are not only the number of candidates but also the root degree r , the degrees of the algebraic coefficient fields, the bit sizes of coefficients, and the expression size of multipliers. Minimal polynomials of combinations of independent algebraic numbers can have degrees approaching the product of the constituent degrees. Expanding β^r and rationalized denominators can sharply increase both coefficient height and expression size. Solving a proper GCD of degree d and forming every phase yields up to rd candidate representations.

Repeated exact certification is itself expensive, so it should be cached and budgeted rather than skipped. Numerical approximations are useful as a *rejection prefilter*: clearly unequal candidates can often be discarded cheaply. They should not be the final acceptance certificate.

9.3 F12: absence of an application-level budget

The outer loop tests its index against a stack whose length may grow. No maximum number of trials, cumulative time limit, memory budget, polynomial degree limit, or total candidate budget is enforced by the program. This does not prove nontermination on every input; it means the caller lacks a predictable resource contract.

Recommended controls include total trials and elapsed time, per-trial limits, maximum candidate-generation size, a cap on allowed polynomial degrees, and an explicit representation-size budget. Exhaustion should return a structured “search incomplete” result together with the best *certified* value found so far. It must not return an unverified intermediate as though it were an exact simplification.

10 Repair strategy and a conservative reference component

10.1 The order of repairs matters

The first repair should preserve the original target, eliminate or guard unsafe preprocessing, and make exact equality an acceptance requirement. Improving speed before these changes merely makes an incorrect result easier to obtain. The next repairs should reject invalid multipliers, guard solver results and empty candidate sets, and restore session hygiene. Only then should the multiplier heuristics be tuned and benchmarked.

For a wrapper over exact algebraic numbers, retain an immutable copy of the original input before any factoring. Each local transformation should preserve that value, and the final output should be certified against it. An exact post-check can prevent a wrong result from escaping, but it does not make unsafe transformations into a good algorithm: a failed certificate should trigger fallback or a structured failure, not a second attempt to justify the altered target.

10.2 Separate value certification from representation quality

Define a target expression language first. A radical-only language might allow rational numbers, algebraic operations, and rational powers, but forbid `Root` and `AlgebraicNumber` as output constructors. Alternatively, an algebraic-canonicalization mode can allow those constructors explicitly. These are different products and should have different success labels.

An example representation score, to be minimized lexicographically, is

$$\text{score}(E) = (\text{forbidden-constructor count}, \text{rdepth}(E), \\ \text{radical-node count}, \text{leaf count}, \text{coefficient bit cost}).$$

This is a proposed engineering policy, not a theorem that one particular score captures all mathematical simplicity. A rewrite is accepted only if it is certified equal to the original and improves the selected score. Merely returning a different expression should not count as improvement.

10.3 A verified single-multiplier trial

Appendix B gives `VerifiedMultiplierTrial`, also supplied as `verified_multiplier_trial.wl`. It deliberately accepts the root degree r as an explicit argument rather than inheriting the source's fragile depth extraction. Its responsibilities are limited:

1. require an exact algebraic target and a nonzero exact algebraic multiplier, with $r \geq 2$;
2. compute the same minimal-polynomial/binomial GCD experiment;
3. enumerate phases and compare candidates to the *original* target using exact algebraic reduction;
4. return a named outcome or a controlled failure, with a per-call time constraint.

It does not claim completeness, minimal radical depth, a full memory budget, or production validation. A result labeled "`VerifiedCandidate`" means only that the returned value passed the exact equality check; an explicit flag reports whether it contains `Root` or `AlgebraicNumber`. The implementation sorts candidate representations before semantic deduplication, so a first, unnecessarily large representative does not defeat the intended leaf-count choice.

The code has been reviewed statically but was not executed in a Wolfram kernel. It is a reference component illustrating the invariants, not a silently substituted and supposedly tested replacement for all 585 lines of the supplied program.

10.4 Queue design and caching

A revised search should keep the candidate generator separate from the trial engine. Use a queue of named records, cache attempts by a certified canonical multiplier key, and check the work budget before generating or enqueueing a batch. Caching by `RootReduce[m]` is one possible policy for supported exact algebraic numbers; the cost of canonicalization itself needs a budget.

Retain unresolved discriminant cofactors as metadata rather than discarding them without a trace. A user can then distinguish “all detected prime combinations exhausted” from “factorization incomplete.” The two statuses have materially different meanings. Changes to search ordering should be benchmarked independently of correctness, with the exact certification layer left unchanged.

11 Reproducibility and regression testing

11.1 Checks actually performed

The archive contains `verify_algebra.py` and its generated `test_results.json`. The script completed **45 independent checks**. They include exact polynomial identities and GCDs for the examples, the negative and complex branch calculations, counterexamples to the unsafe power rules, five denominator inversions with exact remainders, the polynomial-discriminant example, a degree-product example, the meaning of the bad comparator, and exact candidate-count calculations.

One of the 45 checks is explicitly numerical rather than a proof: at 90-digit working precision, 21 combinations of root degree and multiplier verified that the phase enumeration contains a numerical representative of the original target. The proof of that property is Theorem 4.2, not the numerical test. The JSON report distinguishes exact algebra, exact counting, a source-semantics model of the comparator, and this numerical sanity check.

These are *not 45 passing tests of the original program*. The Python script neither executes nor claims to emulate Wolfram’s evaluator, pattern matcher, factorizer, or solver. No runtime benchmark, exact public-wrapper transcript, kernel-specific message sequence, or original-code coverage percentage is claimed.

11.2 Tests prepared for a Wolfram kernel

Appendix C contains `run_wolfram_audit.wl`, which loads the unmodified attachment in a disposable kernel and exercises 14 regression contracts. Some are expected to fail on the original source: their purpose is to expose defects. They check loading side effects, the comparator, correct rationalization, direct and multiplier-assisted successes, original branch preservation, unsafe factorization followed by substitution, zero multiplier handling, an empty seed list, and a basic `alllevels` call.

The runner records the kernel version and restores the original `Print` attributes on normal completion or an ordinary abort. It is not a sandbox against a forcibly terminated process or all possible package side effects. A fresh kernel remains important because the original file clears several global definitions. The proposed reference component should be tested separately; it is not loaded by this original-source regression runner.

11.3 Further tests needed for a release

A release-quality suite should add property-based tests generated from known algebraic targets. Choose a simple exact β , construct $R = \beta^r$, keep all branch metadata, and require any reported answer to equal β exactly. Vary signs and complex arguments, especially near principal-sector and branch cut boundaries. Equality of r th powers is *not* a sufficient oracle.

Test semantically equal but structurally different multipliers, rational and Gaussian-rational denominators, mixed root degrees, near-zero nonzero algebraic numbers, roots represented by `Root`, malformed options, approximate inputs, and exhausted budgets. Test queue interruption and resumption with a mock trial engine so that scheduling invariants can be checked without an expensive field computation. Finally, test idempotence only as a chosen simplification contract: failure to be idempotent can indicate an incomplete normalization policy, but does not by itself prove unequal values.

12 What should not be misdiagnosed

Several unusual details are valid or at least not the defects they might initially appear to be. One-argument `MinimalPolynomial` really does return a callable pure-function representation. The minus sign and possible nonmonic polynomial in the rationalization helper are correct. The roots-of-unity enumeration correctly handles the multiplier phase when the original target is preserved. A numerical sign decision followed by an exact compensating sign does not alone change a product. Boolean non-strict sorting predicates are supported; the actual prime-sort bug is the mistaken slot.

Likewise, a symbolic result from `Element` fails a `PatternTest` rather than automatically being accepted. Several structurally different candidates equal to one target are not several different algebraic values. `Block` is not inherently an error, although it does not provide fresh symbols or restore a separately changed protection attribute. A heuristic returning the unchanged input is not a wrong-value result; it becomes misleading only when presented as proof that no denesting exists.

These distinctions matter because repair effort should target demonstrated failures and missing contracts rather than rewrite every unfamiliar idiom.

13 Conclusion

The program contains a substantial, mathematically motivated experiment: multiplier selection can make a scaled root satisfy a smaller equation over an algebraic coefficient field, and the polynomial-GCD step can expose that fact. Its denominator-rationalization formula is also sound under explicit nonzero algebraicity assumptions.

Its present correctness boundary is inadequate. The original branch can be lost before candidate selection; the custom factorizer uses invalid general power identities; the final zero test is heuristic; and several invalid states are treated as though normal algebraic results were present. The comparator and multiplier-count calculation introduce separate, concrete search defects. Global session changes and implicit state layouts further limit its suitability as reusable software.

The appropriate path forward is therefore *certified heuristic simplification*: preserve the original value, certify every accepted candidate, measure representation improvement separately, and report search limits honestly. Those changes retain the interesting mathematics while removing the assumption that a plausible-looking radical expression must be the same algebraic number.

A Complete supplied source

The following listing is the supplied attachment, with line numbers added by LaTeX. The separate `original.wl` file preserves the original bytes.

```

1 ClearAll[Strad];
2 Unprotect[Print];
3 Strad[eaf_] := Block[{Print = (Null &)}, DenestRadicals3[eaf]];
4 ClearAll[DenestRadicals3];
5 DenestRadicals3[expr_, alllevels_ : False, func_ : denest11] :=
6 Block[{f, mod, level, problems, i1, i2},
7   If[alllevels != True,
8     level = ReplaceAll,
9     level = ReplaceRepeated
10  ];
11  mod = level[Factorc[expr],
12    radicand?AlgebraicQ^pow_Rational := Block[{factored, SumQ, i, j},
13      SumQ[expression_] :=
14        MemberQ[
15          expression, _Plus | Complex[Except[0], _], {0, Infinity}];
16      factored = FactorTermsList[radicand];
17      factored[[2]] =
18        Flatten[{Replace[Factorc[factored[[2]]],
19          prod_Times := List @@ prod]];
20      factored[[1]] =
21        DeleteCases[{#, ComplexExpand[factored[[1]]/#]} &[
22          GCD[Re[factored[[1]]], Im[factored[[1]]]], 1];
23      factored[[1]] = #[[1]]^#[[2]] & /@
24        Flatten[FactorInteger /@ factored[[1]], 1];
25      factored = Flatten[factored];
26      factored =
27        factored //.
28        list : {a___, expr1_, b___, expr2_, c___} := {a, b, c,
29          ComplexExpand[expr1*expr2]} /; And[SumQ[expr1], SumQ[expr2]];
30      j = 1;
31      For[{i = 1}, i <= Length[factored], i++,
32        If[N[Re[factored[[i]]], 100] < 0,
33          factored[[i]] = -factored[[i]];
34          j *= -1
35        ];
36      ];
37      factored = DeleteCases[Prepend[factored, j], 1];
38      f[Simplify[
39        Times @@ (Simplify[RationalizeDenominator[Together[#]]] & /@
40          factored)], pow]
41    ]
42  ];
43  mod = mod //.
44    prod : Times[a___, expr1 : f[rad1_, pow1_], b___,
45      expr2 : f[rad2_, pow2_], c___] :=
46      Times[a, b, c, f[(rad1^pow1)*(rad2^pow2)]] /;
47      And[! FreeQ[rad1, Power], ! FreeQ[rad2, Power]];
48  mod = level[mod, f[rad_, pow_] := f[rad^pow]];
49  mod = mod //.
50    prod : Times[a___, expr1 : f[rad1_], b___, expr2 : f[rad2_],
51      c___] :=
52      Times[a, b, c, f[rad1*rad2]] /;
53      And[And[Re[#] != 0, Im[#] != 0] & [N[rad1, 10]],
54      And[Re[#] != 0, Im[#] != 0] & [N[rad2, 10]]];
55  problems = DeleteDuplicates[Cases[mod, f[_], {0, Infinity}]];
56  Print[problems];
57  Print[];
58  If[alllevels != True,
59    mod =
60      Simplify[
61        mod /. ((# -> Replace[#, f[val_] := func[val]]) & /@ problems)],
62    For[{i1 = 1}, MemberQ[problems, f[_]] === True, i1++,

```



```

63     i1 = Mod[i1, Length[problems], 1];
64     If[MatchQ[problems[[i1]], f[problem_]] /; FreeQ[problem, f[_]],
65       problems[[i1]] = problems[[i1]] -> (problems[[i1]] /. f -> func);
66     For[{i2 = 1}, i2 <= Length[problems], i2++,
67       If[Head[problems[[i2]]] != Rule,
68         problems[[i2]] = problems[[i2]] /. problems[[i1]];
69       ]
70     ];
71   ];
72 ];
73 mod = Simplify[mod /. problems];
74 ];
75 Return[mod];
76 ]
77
78 ClearAll[denest11];
79 denest11[problem_, forcedmultipliers_ : 0] :=
80 Block[{radicand, outerpowers, reductionpower, extrapower,
81   multiplierstack, multipliers, multiplier, terms, history, attempt,
82   solution, updatehistory, stackchange, addmultipliers,
83   trymultiplier, newmultipliers, x, i1, i2, i3},
84   If[! MemberQ[problem,
85     Power[inner_?(Function[var, MemberQ[var, Power[a_, b_], Infinity]]],
86     outer_], {0, Infinity}],
87     Return[problem];
88   ];
89   Print["problem ", problem];
90   reductionpower = (LCM @@ (maxnestdepth[problem,
91     MatchQ[#, Power[_ , _Rational]] &] /. {Power[base_, exp_, _] :>
92     Denominator[exp]}));
93   outerpowers = 1/reductionpower;
94   radicand = problem^reductionpower;
95   extrapower =
96     Times @@ ((LCM @@ #) & /@ (DeleteCases[
97     Cases[nestdepthsort[radicand,
98       MatchQ[#, Power[_ , _Rational]] &],
99     patt : { {expr_, nest_}, Repeated[_ , _], {0, Infinity}}] /;
100     nest >= 2], {Except[Power[_ , _Rational]], _}, {2}] /.
101     patt : {Power[base_, exp_, nest_] :> Denominator[exp]}));
102   Print["outer root: ", reductionpower, " extra root: ", extrapower];
103   Print[];
104
105   (*tries a multiplier
106   returns {<boolean progress>, {multiplier, minpoly, degree}, [answer]}*)
107   trymultiplier[radicand_, outerpowers_, multiplier_, best_,
108     worseknown_] /; MatchQ[outerpowers, Rational[1, _Integer]] :=
109     Block[{minpoly, degree, polygcd, possibilities, result, expected,
110       arg1},
111       minpoly = MinimalPolynomial[(multiplier*radicand)^outerpowers];
112       degree = Exponent[minpoly[arg1], arg1];
113       result = {False, {multiplier, minpoly, degree}};
114       Print["multiplier: ", multiplier, " minpoly degree: ", degree];
115       If[Or[degree < best, GCD[degree, best] < best],
116         polygcd =
117           PolynomialGCD[minpoly[arg1],
118             arg1^(1/outerpowers) - multiplier*radicand,
119             Extension -> Automatic];
120         If[And[Exponent[polygcd, arg1] < (1/outerpowers), polygcd != 1],
121           possibilities =
122             Flatten[Outer[Times,
123               arg1/(multiplier^outerpowers) /. Solve[polygcd == 0, arg1],
124               Exp[2*I*\[Pi]*Range[0, (1/outerpowers) - 1, 1/(1/outerpowers)]]];
125           result =
126             DeleteDuplicates[
127               Select[possibilities, PossibleZeroQ[# - radicand^outerpowers] &]];
128         If[Length[result] > 1,
129           Print[Length[result], " possible solutions, taking the first"];
130           Print[result];

```

```

131     result = Take[result, 1];
132 ];
133 (*progress because a denesting occurred*)
134 result = {True, {multiplier, minpoly, degree}, result[[1]]},
135 If[polygcd != 1,
136   If[
137     Or[degree < best,
138       And[GCD[degree, best] < best, worseknown != True]],
139     (*progress because degree of minpoly has decreased*)
140     result = {True, {multiplier, minpoly, degree}}
141   ],
142   Print[
143     "PolynomialGCD error: ", {radicand, outerpower, multiplier,
144       minpoly}]
145   ]
146 ],
147 If[And[worseknown != True, degree != best],
148   (*progress because this multiplier has greater degree than a pre\
149 vious multiplier*)
150   Print["previous progress identified"];
151   result = {True, {multiplier, minpoly, degree}};
152   ]
153 ];
154 Return[result];
155 ];
156
157 (*form for attempts is {multiplier, minpoly, degree},
158 attempt2 is Null if only one attempt is available*)
159 newmultipliers[smaller_, larger_, reductionpower_] :=
160 Block[{done, new, primes, multiplierprimes, minprimes, maxprimes,
161   primeratio, multipliercap, arg1},
162   minprimes = 5;
163   maxprimes = 14;
164   primeratio = 100;
165   multipliercap = 1000;
166
167   (*two possibilites: reduction in degree,
168   reduction only in GCD of degrees*)
169   If[Or[larger == Null,
170     GCD[smaller[[-1]], larger[[-1]]] ==
171     Min[smaller[[-1]], larger[[-1]]],
172     (*one degree is a multiple of the other, usual case*)
173     primes =
174     DeleteCases[
175       FactorInteger[Discriminant[smaller[[2]][arg1], arg1],
176         Automatic], {-1, 1}];
177     primes =
178     Sort[DeleteCases[primes,
179       factor : {prime_, _} /;
180       And[! PrimeQ[prime], (Print["composite factor: ", factor];
181         True)], #1[[1]] <= #2[[1]] &];
182     primes = Flatten[primes[[All, 1]]];
183     multiplierprimes = primes;
184     For[{i3 = Length[primes]},
185       And[i3 > minprimes,
186         multiplierprimes[[i3]]/multiplierprimes[[i3 - 1]] > primeratio],
187       i3--,
188       multiplierprimes[[i3]] = 0;
189     ];
190     multiplierprimes =
191     DeleteCases[multiplierprimes, 0][[
192       1 ;; Min[
193         Max[minprimes, Ceiling[Log[multipliercap]/Log[reductionpower]]],
194         Length[multiplierprimes]]]];
195     new =
196     smaller[[1]]*
197     Drop[Union[
198       Divisors[(Times @@ multiplierprimes)^(reductionpower - 1)],

```

```

199     primes], 1];
200     ,
201     (*different degree reductions in each degree from a larger degree\
202 *)
203     Print[
204         "GCD of smallest degrees is smaller than any observed degree: \
205 ", {smaller[{{1, -1}}], larger[{{1, -1}}]};
206     new = smaller[[1]]*larger[[1]]*{1};
207     Print[PolynomialGCD @@ {smaller[[2]][x], larger[[2]][x]};
208     ];
209     new = {{False, smaller}, ComplexitySort[new]};
210     Return[new];
211     ];
212
213 (*compute initial set of multipliers*)
214 If[ListQ[forcedmultipliers],
215     multipliers = {{False, {0, Infinity, Infinity}}, forcedmultipliers},
216     (*old, may be better*)
217     (*
218     terms=DeleteCases[Replace[#, {Times[a___,
219     Alternatives[_Rational, _Integer], b___]\[RuleDelayed]a*b,
220     Alternatives[_Rational, _Integer]\[Rule]0, Complex[a_,
221     b_]\[RuleDelayed]Sign[b]*I]}&/@ (List@@Expand[
222     RationalizeDenominator[radicand]]), 0];
223     *)
224     terms =
225     DeleteCases[
226     Replace[#, {Times[a___, Alternatives[_Rational, _Integer],
227     b___] :> a*b, Alternatives[_Rational, _Integer] -> 0,
228     Complex[a_, b_] :> Sign[b]*I]} & /@ (List @@
229     Expand[RationalizeDenominator[radicand]]), 0];
230     multipliers = Prepend[Replace[If[Head[#] == Times,
231
232     Times @@ (Function[{var},
233     var^(maxnestdepth[var,
234     Function[{var2},
235     MatchQ[var2, Power[_ , _Rational]]][[
236     1]] /. {{Power[base_,
237     exp_], _} :> (Denominator[exp] - Numerator[exp])/
238     Numerator[exp], {Complex[
239     0, _], _} :> -1}]] /@ (List @@ #)),
240     #^(maxnestdepth[#,
241     Function[{var}, MatchQ[var, Power[_ , _Rational]]][[
242     1]] /. {{Power[base_,
243     exp_], _} :> (Denominator[exp] - Numerator[exp])/
244     Numerator[exp], {Complex[0, _], _} :> -1})
245     ],
246     Times[a___, Alternatives[_Rational, _Integer], b___] :>
247     a*b] & /@ Sort[terms, (Re[#1] <= Re[#2]) &], 1];
248     multipliers = {{True, {0, Infinity, Infinity}}, multipliers};
249     ];
250
251     solution = problem;
252     history = {{0, Infinity, Infinity}};
253     (*multiplierstack form {{<do full set>,{<multiplier,minpoly,
254     degree for generating this set>},{<multipliers>}},...}*)
255     multiplierstack = {multipliers};
256     (*add power set of multipliers as second element in multiplierstack*)
257     (*multiplierstack=Append[multiplierstack,
258     {multiplierstack[[1,1]],ComplexitySort[DeleteCases[Union[(Times@@#)&/@
259     Subsets[DeleteCases[multipliers[[2]],1]]],mult_/_;MemberQ[
260     multipliers[[2]],mult]]]}];*)
261     Catch[
262     For[{i1 = 1}, i1 <= Length[multiplierstack], i1++,
263     (*compute new multipliers from data if not already done*)
264     If[ListQ[multiplierstack[[i1, 2]]] != True,
265     multiplierstack[[i1]] =
266     multiplierstack[[i1, -2]] @@ (multiplierstack[[i1, -1]]);

```

```

267 ];
268 multipliers = multiplierstack[[i1]];
269 stackchange = False;
270 If[multipliers[[1, 2]] != history[[1]],
271   Print["multiplier count: ", Length[multipliers[[2]]], " from: ",
272     multipliers[[1, 2, {1, -1}]]],
273   If[! ListQ[forcedmultipliers],
274     Print["multiplier count: ", Length[multipliers[[2]]],
275       " multipliers: ", multipliers[[2]]];
276     Print["multiplier count: ", Length[multipliers[[2]]];
277   ]
278 ];
279 For[{i2 = 1}, i2 <= Length[multipliers[[2]]], i2++,
280   multiplier = multipliers[[2, i2]];
281   updatehistory = False;
282   addmultipliers = False;
283   (*try a multiplier, different based on history*)
284   If[And[Length[history] >= 3, history[[-2]] != history[[1]],
285     attempt =
286       trymultiplier[radicand, outerpower, multiplier,
287         history[[-1, -1]], True],
288     attempt =
289       trymultiplier[radicand, outerpower, multiplier,
290         history[[-1, -1]], False];
291   ];
292   multiplierstack[[i1, 2, i2]] = {multiplier, attempt[[2, -1]]};
293
294   If[And[multipliers[[1, 1]] == True,
295     attempt[[2, -1]] <= history[[-1, -1]]],
296     addmultipliers = True
297   ];
298   (*check if progress has been made on this attempt,
299   no action taken if no progress*)
300   If[attempt[[1]] == True,
301     (*history will be updated*)
302     updatehistory = True;
303
304     (*decide if a stackchange will be made*)
305     If[history[[-1]] != history[[1]],
306       stackchange = True;
307     ];
308
309   If[ListQ[attempt[[-1]]] != True,
310     (*a solution was obtained*)
311     Print["success: ",
312       Join[attempt[[2, {1, -1}]],
313         If[IntegerQ[multiplier], {FactorInteger[multiplier]}, {}]]];
314     solution = Simplify[attempt[[-1]]];
315     If[
316       Or[history[[-1]] == history[[1]],
317         history[[2, -1]]/attempt[[2, -1]] >= reductionpower],
318       (*not worth trying for additional denesting, exit immediately*)
319       history = Append[history, attempt[[2]]];
320       Throw["solution"],
321       (*further denesting will be attempted*)
322       Print["attempting additional denesting, current solution is:"];
323       Print[solution];
324     ];
325   ];
326
327   If[
328     And[history[[-1]] != history[[1]],
329       Or[attempt[[2, -1]] < history[[-1, -1]],
330         history[[-2]] == history[[1]]],
331     (*improvement has occurred, new multipliers will be created*)
332     Print["improvement ", attempt[[2, {1, -1}]]];
333     addmultipliers = True;
334   ];

```

```

335 ];
336
337 (*add new multipliers if needed*)
338 If[addmultipliers === True,
339   Block[{smaller, larger, done, new},
340     If[attempt[[2, -1]] <= history[[-1, -1]],
341       smaller = attempt[[2]];
342       larger = history[[-1]],
343       smaller = history[[-1]];
344       larger = attempt[[2]];
345     ];
346     new = {{False, smaller},
347           newmultipliers, {smaller,
348             If[larger != history[[1]], larger, Null], reductionpower}};
349
350   Print["adding new multipliers from: ", multiplier];
351   (*update multiplierstack with new multipliers*)
352   If[multipliers[[1, 1]] === False,
353     (*use new multipliers immediately*)
354     done = {multipliers[[1]], multipliers[[2, 1 ;; i2]]};
355     multiplierstack = Insert[multiplierstack, new, i1 + 1];
356     If[i2 < Length[multipliers[[2]]],
357
358       multiplierstack =
359         Insert[multiplierstack, {multipliers[[1]],
360           multipliers[[2, i2 + 1 ;; -1]]}, i1 + 2];
361     ];
362     multiplierstack[[i1]] = done,
363     (*use new multipliers later*)
364     stackchange = False;
365     For[{i3 = Length[multiplierstack]}, i3 >= i1, i3--,
366
367       If[Or[multiplierstack[[i3, 1, 2]] === history[[1]],
368         new[[1, 2, -1]] > multiplierstack[[i3, 1, 2, -1]],
369         And[new[[1, 2, -1]] === multiplierstack[[i3, 1, 2, -1]],
370         LeafCount[new[[1, 2, 1]]] >=
371         LeafCount[multiplierstack[[i3, 1, 2, 1]]]],
372       multiplierstack = Insert[multiplierstack, new, i3 + 1];
373       Break[];
374     ];
375   ];
376 ];
377 ];
378 ];
379
380 (*update history if needed*)
381 If[updatehistory === True,
382   For[{i3 = Length[history]}, i3 >= 1, i3--,
383     If[history[[i3, -1]] > attempt[[2, -1]],
384       history = Insert[history, attempt[[2]], i3 + 1];
385       Break[];
386     ];
387   ];
388 ];
389
390 (*break inner loop if stack has changed*)
391 If[stackchange === True,
392   Break[];
393 ];
394 ];(*end of inner loop*)
395
396 (*add additional multipliers after first pass (if no forced multipliers)*)
397 If[And[i1 === 1, ! ListQ[forcedmultipliers]],
398   Block[{best, worst, extra, f},
399     best = Min[multiplierstack[[i1, 2, All, 2]]];
400     worst = Max[multiplierstack[[i1, 2, All, 2]]];
401     extra =

```

```

403 DeleteCases[
404   DeleteDuplicates[
405     Replace[(Distribute[
406       f @@ (#^Range[0, 1] & /@ (Cases[
407         multiplierstack[[i1, 2]], {mult_, degree_} /;
408         degree < worst][[All, 1]])),
409       List] /. {f[a___, Power[base_Times, exp_], b___] :>
410         f[a, b, Sequence @@ ((List @@ base)^exp)],
411       f[a___, prod_Times, b___] :> f[a, b, Sequence @@ prod],
412       f[a___, Power[base_, exp_], b___] :>
413       f[a, b, (-1)^exp, (-base)^exp] /;
414       And[base != -1, Re[base] < 0],
415       f[a___, Power[base_, Rational[num1_, denom_]], b___,
416       Power[base_, Rational[num2_, denom_]], c___] :>
417       f[a, b, c, base^(Mod[num1 + num2, denom]/denom)]} /.
418       f -> Times, Times[a___, _Integer, b___] :> a*b, {1}] /.
419       Power[base_, Rational[a_, b_]] :> base^(Mod[a, b]/b)],
420     Alternatives @@ multiplierstack[[i1, 2, All, 1]]];
421   If[Length[extra] >= 1,
422     Print["adding multipliers based on progress in initial guess"];
423     multiplierstack =
424       Insert[multiplierstack, {multiplierstack[[i1, 1]], extra},
425       i1 + 1];
426   ];
427 ]
428 ];
429
430 ](*end of outer loop*),
431 "solution"
432 ](*end of Catch, used to break the loop when a solution is found*)
433
434 Print["history: ", #[[{1, -1}]] & /@ history];
435 If[solution != problem,
436   Print[solution],
437   Print["no progress"];
438 ];
439 Print[];
440 Return[solution];
441 ]
442
443 ClearAll[RationalizeDenominator];
444 RationalizeDenominator[expr_] :=
445   Block[{conjugate, denom, x},
446     denom = Denominator[expr];
447     conjugate = MinimalPolynomial[denom];
448     Expand[
449       Numerator[expr]*PolynomialQuotient[conjugate[x], x - denom, x] /.
450       x -> 0]/(-1*conjugate[0])
451   ]
452
453 ClearAll[AlgebraicQ];
454 AlgebraicQ[num_] := Element[num, Algebraics]
455
456 ClearAll[nestdepths]
457 nestdepths[expr_, func_] :=
458   Block[{inner},
459     If[Depth[expr] === 1,
460       {expr, If[func[expr], 1, 0]},
461       inner = Map[nestdepths[#, func] &, (List @@ expr), {1}];
462       {expr,
463         Sequence @@
464         inner, ((Max @@ ({#[[-1]]} & /@ inner)) + If[func[expr], 1, 0])}
465     ]
466   ]
467
468 ClearAll[nestdepthsort]
469 nestdepthsort[expr_, func_] :=
470   Block[{data},

```

```

471 data = nestdepths[expr, func];
472 data =
473   data //.
474   patt : {a : Except[_List], b : __, c : Except[_List]} :> {a, c,
475     b};
476 data = data //. patt : {rep : Repeated[_], _} :> Sequence @@ patt;
477 data = Gather[{data}, (#1[[2]] == #2[[2]]) &];
478 data = Sort[data, (#1[[1, 2]] >= #2[[1, 2]]) &];
479 data
480 ]
481
482 ClearAll[maxnestdepth]
483 maxnestdepth[expr_, func_] :=
484   Block[{data, max},
485     data = nestdepths[{expr}, func][[2]];
486     max = data[[-1]];
487     data =
488       FixedPoint[
489         DeleteCases[# /. {a_, b_} :> {} /; b < max, {}, Infinity] &, data];
490     data =
491       If[And[ListQ[#], Length[#] > 2], Sequence @@ (#[[2 ;; -2]]), #] & /@
492       data;
493     If[Length[data] > 2,
494       data = data[[2 ;; -2]],
495       data = {data}
496     ];
497     data
498   ]
499
500 ClearAll[ComplexitySort];
501 ComplexitySort[expr_] :=
502   Return[Sort[expr,
503     Switch[LeafCount[#1] - LeafCount[#2], a_ /; a < 0, True,
504       a_ /; a > 0, False, 0, Abs[#1] <= Abs[#2]] &]];
505
506 ClearAll[Factorc]
507 Factorc[expr_] :=
508   Flatten[
509     fullfactor[
510       expr //. {pow : {a : Except[_List], b : Except[_List]} :> {a^b},
511       prod : {rep : (Repeated[{Except[_List]], {2,
512         Infinity}})} :> {Times[rep]},
513       sum : {rep : Repeated[{Except[_List]], {2, Infinity}}} :>
514       Plus[rep],
515       pow2 : {a : Except[_List], b : Except[_List]} :> {a^b},
516       pow3 : {{a : Except[_List]}, b : Except[_List]} :> {a^b}}][[1]]
517
518 ClearAll[fullfactor]
519 fullfactor[expr_] :=
520   Block[{factorlist, i = 0, j = 0, done = False},
521     factorlist = FactorList[expr];
522     While[! done && j < 5,
523       j++;
524       done = True;
525       For[{i = 1}, i <= Length[factorlist], i++,
526         Which[
527           Head[factorlist[[i, 1]]] === Power,
528           done = False;
529           factorlist[[
530             i]] = {{factorlist[[i, 1, 1]],
531             factorlist[[i, 1, 2]]*factorlist[[i, 2]]},
532           And[
533             MemberQ[{Integer, Rational}, Head[factorlist[[i, 1]]], !
534             PrimeQ[factorlist[[i, 1]], Abs[factorlist[[i, 1]]] != 1],
535           done = False;
536           factorlist[[i]] = {#[[1]], #[[2]]*factorlist[[i, 2]]} & /@
537             FactorInteger[factorlist[[i, 1]],
538             Head[factorlist[[i, 1]]] === Plus,

```

```

539     done = False;
540     factorlist[[i]] =
541       combine[fullfactor /@
542         Replace[factorlist[[i, 1]], Plus[a_, b_] := {a, b}],
543       factorlist[[i, 2]]],
544     True,
545     factorlist[[i]] = {factorlist[[i]]}
546   ]
547 ];
548 factorlist = Flatten[factorlist, 1]
549 ];
550 factorlist
551 ]
552
553 ClearAll[combine];
554 combine[terms_List, pow_] :=
555 Block[{simpler = terms, base, bases, positions, factor, i, j, k},
556   For[{i = 1}, i <= Length[terms], i++,
557     base = terms[[i]];
558     For[{j = 1}, j < Length[base], j++,
559       For[{k = j + 1}, k <= Length[base], k++,
560         If[base[[j, 1]] == base[[k, 1]],
561           base[[j]] = {base[[j, 1]], base[[j, 2]] + base[[k, 2]]};
562           base[[k]] = {0, 0}
563         ]
564       ];
565       If[base[[j, 1]] == 1,
566         base[[j]] = {0, 0}
567       ]
568     ];
569     simpler[[i]] = Delete[base, Position[base, {0, 0}]];
570   ];
571   bases = Intersection @@ Map[#[[1]] &, simpler, {2}];
572   For[{i = 1}, i <= Length[bases], i++,
573     positions =
574       Position[#, {bases[[i]], Except[_List]}, 1] & /@ simpler;
575     factor =
576       Min[Extract[simpler[[#]], positions[[#]]][[1, 2]] & /@
577         Range[1, Length[simpler], 1]];
578     For[{j = 1}, j <= Length[simpler], j++,
579       simpler[[j, Sequence @@ (positions[[j, 1]]), 2]] -= factor
580     ];
581     bases[[i]] = {bases[[i]], factor*pow};
582   ];
583   Append[bases, {simpler, pow}]
584 ]

```


B Defensive single-trial reference code

This reference component is not a complete denester and was not executed in a Wolfram kernel during the review. It deliberately calls a certified value a “candidate,” not a certified reduction of radical depth.

```

1 (* A conservative, single-multiplier reference component for the article.
2   This is NOT a complete replacement denester, and was not executed in a
3   Wolfram kernel during this review. The original target is never discarded.
4   Load this file separately; it does not load or modify the supplied code. *)
5 BeginPackage["RadicalAudit"];
6 VerifiedMultiplierTrial::usage =
7   "VerifiedMultiplierTrial[beta, r, m] tries one nonzero exact algebraic multiplier, preserving the
8   original branch. A verified candidate is not a certificate of radical-depth reduction.";
9
10 ClearAll[exactAlgebraicValueQ, exactEqualQ];
11 exactAlgebraicValueQ[z_] :=
12   TrueQ[NumericQ[z] && Precision[z] === Infinity &&
13     Element[z, Algebraics]];
14 exactEqualQ[a_, b_] :=
15   TrueQ[Quiet[Check[RootReduce[a - b] === 0, False]]];
16
17 Options[VerifiedMultiplierTrial] = {TimeConstraint -> 30};
18 VerifiedMultiplierTrial[beta_, r_, m_, OptionsPattern[]] :=
19   Module[{limit = OptionValue[TimeConstraint]},
20     If[!(limit === Infinity ||
21       TrueQ[NumericQ[limit] && Im[limit] == 0 && limit > 0]),
22       Return[Failure["InvalidTimeConstraint", <||>]]];
23     TimeConstrained[
24       Quiet[Check[
25         Module[{x, radicand, scaled, minpoly, g, d, rules,
26           roots, phases, candidates, valid, best},
27           If[!TrueQ[IntegerQ[r] && r >= 2],
28             Return[Failure["InvalidRootDegree", <||>]]];
29           If[!exactAlgebraicValueQ[beta] || !exactAlgebraicValueQ[m],
30             Return[Failure["ExactAlgebraicInputsRequired", <||>]]];
31           If[exactEqualQ[m, 0],
32             Return[Failure["ZeroMultiplier", <||>]]];
33           If[exactEqualQ[beta, 0],
34             Return[<|"Status" -> "Unchanged", "Value" -> 0,
35               "Verified" -> True|>]];
36           radicand = beta^r;
37           scaled = m radicand;
38           minpoly = MinimalPolynomial[scaled^(1/r), x];
39           If[!PolynomialQ[minpoly, x],
40             Return[Failure["MinimalPolynomialFailed", <||>]]];
41           g = PolynomialGCD[minpoly, x^r - scaled,
42             Extension -> Automatic];
43           d = Exponent[g, x];
44           If[!PolynomialQ[g, x] || !IntegerQ[d],
45             Return[Failure["PolynomialGCDFailed", <||>]]];
46           If[!(1 <= d < r),
47             Return[<|"Status" -> "NoProperGCD", "Value" -> beta,
48               "GCDDegree" -> d|>]];
49           rules = Solve[g == 0, x];
50           If[!ListQ[rules] || rules === {} ||
51             !TrueQ[And @@ (MatchQ[#, {_Rule}] & /@ rules)],
52             Return[Failure["UnexpectedSolveResult", <||>]]];
53           roots = x /. rules;
54           phases = Table[Exp[2 Pi I k/r], {k, 0, r - 1}];
55           candidates = Flatten[
56             Outer[Times, roots/m^(1/r), phases], 1];

```

```

57     valid = Select[candidates,
58         exactAlgebraicValueQ[#] && exactEqualQ[#, beta] &];
59     valid = SortBy[valid, LeafCount];
60     valid = DeleteDuplicates[valid, exactEqualQ];
61     If[valid === {},
62         Return[Failure["NoCertifiedCandidate", <|>]]];
63     best = First[valid];
64     <|"Status" -> "VerifiedCandidate", "Original" -> beta,
65         "Value" -> best, "Verified" -> True,
66         "Multiplier" -> m, "GCDDegree" -> d,
67         "ContainsRootObjects" ->
68             !FreeQ[best, _Root | _AlgebraicNumber]|>
69     ],
70     Failure["EvaluationFailed", <|>]],
71     limit, Failure["TimeLimit", <|>]]
72 ];
73 End[];
74 EndPackage[];

```

C Unexecuted Wolfram regression contracts

Run these tests on the original source in a fresh kernel. Some tests are expected to fail; they express desired correctness and error-handling contracts, not a claim that the original implementation satisfies them.

```

1 (* Regression contracts for the UNMODIFIED supplied source.
2   NOT EXECUTED in a Wolfram kernel during this review.
3   Some tests are EXPECTED TO FAIL: they expose defects, rather than documenting
4   correct existing behavior. Run only in a fresh/disposable kernel because
5   original.wl clears global definitions and unprotects System`Print.
6   Invocation: wolframscript -file run_wolfram_audit.wl
7   The original Print attributes are restored on normal completion or Abort. *)
8 ClearAll[auditEqualQ, auditEval];
9 auditEqualQ[a_, b_] :=
10   TrueQ[Quiet[Check[RootReduce[a - b] === 0, False]]];
11 SetAttributes[auditEval, HoldAll];
12 auditEval[expr_] :=
13   TimeConstrained[Block[{$Output = {}}, Quiet[expr]], 20, $TimedOut];
14
15 Module[{dir, savedPrintAttributes, report, protectionOK},
16   dir = DirectoryName[$InputFileName];
17   savedPrintAttributes = Attributes[Print];
18   CheckAbort[
19     Get[FileNameJoin[{dir, "original.wl"}]];
20     protectionOK = MemberQ[Attributes[Print], Protected];
21     report = TestReport[{
22       VerificationTest[protectionOK, True,
23         TestID -> "loading-preserves-Print-protection"],
24       VerificationTest[(#1[[1]] <= #2[[2]] &)[{2, 1}, {3, 1}], True,
25         TestID -> "prime-comparator-orders-2-before-3"],
26       VerificationTest[
27         auditEqualQ[RationalizeDenominator[1/(1 + Sqrt[2])],
28           Sqrt[2] - 1], True, TestID -> "rationalizer-real"],
29       VerificationTest[
30         auditEqualQ[RationalizeDenominator[1/(1 + I)], (1 - I)/2],
31         True, TestID -> "rationalizer-complex"],
32       VerificationTest[
33         auditEqualQ[auditEval[denest11[Sqrt[3 + 2 Sqrt[2]]], {1}]],
34         1 + Sqrt[2]], True, TestID -> "direct-denesting-success"],
35       VerificationTest[
36         auditEqualQ[auditEval[denest11[Sqrt[5 + 2 Sqrt[6]]], {2}]],
37         Sqrt[2] + Sqrt[3]], True,
38         TestID -> "rational-multiplier-success"],
39       VerificationTest[
40         auditEqualQ[auditEval[denest11[-Sqrt[3 + 2 Sqrt[2]]], {1}]],
41         -1 - Sqrt[2]], True, TestID -> "negative-original-branch"],
42       VerificationTest[
43         auditEqualQ[auditEval[denest11[
44           Sqrt[-1 + 2 I Sqrt[2]] Sqrt[-2 + 2 I Sqrt[3]], {1}]],
45         (1 + I Sqrt[2]) (1 + I Sqrt[3])],
46         True, TestID -> "grouped-complex-original-branch"],
47       VerificationTest[
48         auditEqualQ[auditEval[DenestRadicals3[
49           Sqrt[-1 + 2 I Sqrt[2]] Sqrt[-2 + 2 I Sqrt[3]]],
50         (1 + I Sqrt[2]) (1 + I Sqrt[3])],
51         True, TestID -> "public-wrapper-complex-product"],
52       VerificationTest[
53         Module[{t}, auditEqualQ[
54           auditEval[Factorc[Sqrt[t^2]]] /. t -> -2, 2]],
55         True, TestID -> "nested-power-symbolic-branch"],
56       VerificationTest[
57         Module[{t}, auditEqualQ[

```

```

58     auditEval[Factorc[Sqrt[t + t^2]] /. t -> -2, Sqrt[2]],
59     True, TestID -> "factored-sum-symbolic-branch"],
60 VerificationTest[
61     MatchQ[auditEval[denest11[Sqrt[3 + 2 Sqrt[2]], {0}]], _Failure],
62     True, TestID -> "zero-multiplier-controlled-rejection"],
63 VerificationTest[
64     auditEqualQ[auditEval[denest11[Sqrt[3 + 2 Sqrt[2]], {}]],
65     Sqrt[3 + 2 Sqrt[2]], True, TestID -> "empty-seed-list-no-change"],
66 VerificationTest[
67     auditEqualQ[auditEval[DenestRadicals3[
68     Sqrt[3 + 2 Sqrt[2]], True]], 1 + Sqrt[2]],
69     True, TestID -> "alllevels-basic-correctness"]
70 ];
71 Attributes[Print] = savedPrintAttributes;
72 Print[report];
73 Print["Kernel version: ", $Version];,
74 Attributes[Print] = savedPrintAttributes;
75 Abort[]
76 ]
77 ];

```

References

- [1] Supplied attachment, *Pasted text.txt*, reproduced as `original.wl`; 585 physical lines, 21,199 bytes. Source-line citations in this article refer to this file. No authorship, date of original composition, or supported Wolfram version was supplied.
- [2] Wolfram Research, *Power*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Power.html>.
- [3] Wolfram Research, *MinimalPolynomial*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/MinimalPolynomial.html>.
- [4] Wolfram Research, *PolynomialGCD*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/PolynomialGCD.html>.
- [5] Wolfram Research, *PossibleZeroQ*, Wolfram Language documentation, including the exact-algebraics method and documented false positive examples. <https://reference.wolfram.com/language/ref/PossibleZeroQ.html>.
- [6] Wolfram Research, *RootReduce*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>.
- [7] Wolfram Research, *FactorInteger*, Wolfram Language documentation, including the partial-factorization `Automatic` argument. <https://reference.wolfram.com/language/ref/FactorInteger.html>.
- [8] Wolfram Research, *Sort*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Sort.html>.
- [9] Wolfram Research, *Function*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Function.html>.
- [10] Wolfram Research, *Block*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Block.html>.
- [11] Wolfram Research, *Module*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Module.html>.
- [12] Wolfram Research, *Unprotect*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Unprotect.html>.
- [13] Wolfram Research, *ReplaceRepeated*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ReplaceRepeated.html>.
- [14] Wolfram Research, *FactorList*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/FactorList.html>.
- [15] Wolfram Research, *Element*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Element.html>.
- [16] Wolfram Research, *PatternTest*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/PatternTest.html>.

- [17] Wolfram Research, *ComplexExpand*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ComplexExpand.html>.
- [18] Wolfram Research, *Solve*, Wolfram Language documentation, including `Cubics` and default algebraic-root representations. <https://reference.wolfram.com/language/ref/Solve.html>.

All external documentation above was consulted on September 5, 2026. It establishes language semantics; the source-specific conclusions, proofs, and proposed changes are the analysis in this article.